

FRED TALKS

29th June 2026

CONTENTS

How agents are transforming work | OpenAI

OPENAI · 1582 WORDS

LLM Judges aren't the shortcut you think

DOUG TURNBULL · 1869 WORDS

Why I don't like the "staff engineer archetypes"

SEANGOEDECKE.COM RSS FEED · 1426 WORDS

Giving LLMs a personality is just good engineering

SEANGOEDECKE.COM RSS FEED · 1062 WORDS

How agents are transforming work | OpenAI

OPENAI · 28 JUN 2026 · [SOURCE](#)

June 25, 2026

[Company](#)

A new Economic Research paper measuring Codex's economic potential at the frontier.

[Read the paper](#)

Listen to article

[AUDIO OMITTED]

Share

Agentic AI changes the unit of knowledge work from single interactions to delegated, long-horizon tasks. Chatbot interactions are often short and self-contained. Agents can operate independently for minutes or hours while orchestrating tool calls, interacting with environments, and iterating towards solutions. As a result, agents are quickly becoming the most powerful AI tool for work.

Over the last year, we witnessed this transformation first-hand at OpenAI. For the first few months after Codex was released to the public, ChatGPT remained the default AI tool for work within OpenAI. Through August 2025, the average OpenAI worker spent less than 10% of their tokens on Codex. Now, every department, including non-technical departments such as Legal and Recruiting, uses Codex as their primary AI tool for work. This pattern reflects what we believe will be the future of work given the expanded capabilities and accessibility of agentic tools.

Some content could not be imported from the original document. [View content ↗](#)

Some content could not be imported from the original document. [View content ↗](#)

Codex adoption grew in tandem with Codex’s capabilities. As Codex leveraged stronger models and new product features, it became capable of taking on an expanding set of productive tasks. Across Individual users, Organizational users, and OpenAI workers, we document four trends over the past year:

- **People use Codex for longer-horizon work.** By May 2026, 80.6% of sampled individual users made at least one Codex request estimated to exceed 30 minutes of human work, 70.2% made one estimated to exceed one hour, and 25.6% made at least one Codex request estimated to exceed eight hours.
- **Codex became the primary AI tool for every department at OpenAI.** Engineering moved first, but Legal, Finance, and Recruiting crossed into Codex being their primary AI tool around April 2026. For the average OpenAI worker, Codex usage now accounts for more than 85% of output tokens. Since Codex users tend to use more tokens than non-users, its share of overall tokens is even higher: Codex accounts for 99.8% of weekly output tokens generated within OpenAI.
- **Non-developer adoption grew especially rapidly, outpacing developer adoption.** Since August 2025, non-developer users rose 137x for individual users, 189x for organizational users, and 12x within OpenAI.
- **Codex enabled OpenAI workers to do tasks outside their job description.** While technical usage is still most prevalent among engineers, non-technical users regularly use Codex to take on coding or technical execution, including automation, data transformation, tooling, debugging, and structured analysis.

Agents work longer hours on harder tasks

Nearly a quarter of all Codex requests are for tasks that would take a person more than one hour to complete⁽¹⁾. As Codex’s capacity for independent long-context work improved, users shifted from short interactions toward more difficult tasks with longer horizons.

The chart below estimates the share of individual users that crossed four human-time thresholds: tasks that would take a person more than 30 minutes, more than one hour, more than four hours, and more than eight hours⁽²⁾. From December 2025 to May 2026, the share of users who made a request that was estimated to correspond to work that would take a person more than 30 minutes rose to 80.6%. The share making a request that would take a person more than one hour rose to 70.2%. The share requesting work that would take a person more than eight hours grew the fastest from a low base.

The growth in agentic usage can be seen in daily Codex runtime. Among daily active users at OpenAI, the heaviest users ask Codex to run many hours of agent work in a single day. By June 2026, users at the 99th percentile regularly generated more than 60 hours of Codex agent turns per day, distributed across multiple, parallel agents. As Codex became more powerful and parallelizable, users moved from only asking Codex for one answer at a time into increasingly orchestrating multiple agent tasks over the course of a day.

Adoption continues to move from engineers to the rest of OpenAI

Engineers at OpenAI began adopting Codex first, gradually. The average engineer at the company shifted the majority of their usage of OpenAI products to Codex by December 2025. Today, the average engineer generates 99% of their output tokens with Codex rather than ChatGPT. Legal, finance, and recruiting crossed over to majority use of Codex later, around April 2026, but their transitions were much faster. The average lawyer or recruiter at OpenAI now generates more than 85% of their output tokens on Codex.

Over the last six months, Codex usage has deepened and intensified at OpenAI. Among active internal users, change in combined output tokens rose sharply across departments. Research saw the biggest jump: by June 2026, median use was 56 times higher than in November 2025. Customer Support rose 32 times and Engineering rose 27 times, while Legal grew more gradually but still reached 13 times its November level.

These two patterns together illustrate how Codex has transformed how OpenAI uses AI to do productive work. Across the company, users are switching from chatbots to agents as their primary form of AI interaction and are deploying an exponentially growing amount of agentic labor.

Non-developers are the fastest-growing user groups

Across all user groups—OpenAI, organizational, and individual users—Codex use began with developers, the natural target audience for what began as a coding tool. However, as Codex expanded toward more general knowledge work, adoption among non-developers grew even more quickly. As shown in the user growth chart below, weekly non-developer users rose faster than developer users among individual, organizational, and OpenAI populations. By early June 2026, non-developer individual users multiplied 137 times since August 2025. Non-developer organizational users increased 189-fold, and non-developer OpenAI users increased 12-fold, likely because this group already started at a well above average starting point.

The shift does not mean every non-developer is using Codex in the same way as an engineer. Rather, it means that more non-developers are using Codex for some kind of agentic work.

Codex is expanding the horizon of potential productive work

Codex enables non-technical departments to accelerate workflows previously bottlenecked by technical expertise. The heat map below compares inferred occupations within OpenAI to the type of work represented in Codex outputs. Engineering and coding show up as the largest category for data science and research, whereas knowledge work is the largest category for finance and business operations, marketing, operations, and other departments.

That said, agentic tools can expand what an individual worker can do. For instance, over one-fourth of work done with Codex by workers in business functions was engineering or coding. Agents can lower the cost of moving across task boundaries and help workers do adjacent work that used to require more specialized technical support.

Occupation vs. work done with Codex

Consolidated inferred department	Work category				
	Engineering / Coding	Data Analysis	Financial Analysis	Knowledge Work	Other
Engineering	72%	4%	1%	18%	5%
Data Science / Research	51%	10%	0%	30%	9%
Finance / Biz Ops	31%	9%	16%	34%	10%
Product / Marketing / Ops	25%	3%	7%	51%	15%
Other	50%	7%	2%	38%	4%

Share of output tokens within department

What this means for agents' economic potential

Increased use of agentic tools by non-engineer employees expands the frontier of what these workers can do. That matters for businesses deciding how to redesign workflows, for employees learning which skills become more valuable, and for policymakers and researchers trying to understand how AI changes the labor market.

Our paper shows how frontier users adopt capable agentic tools at the frontier. Our results demonstrate what unfolds when people have broad, low-friction access to capable agentic tools: as the tools improve, people use them for longer, more complex, and more cross-functional work. As time goes on, this is likely to be what the future of work looks like.

- [2026](#)
- [Economic Research](#)
- [Codex](#)

Author

OpenAI

Footnotes

1. Task horizon is estimated using an LLM-as-judge with access to Codex transcripts.

[SVG OMITTED]

[\[find in text\]](#)
2. These thresholds are model-estimated, so they should be treated as directional rather than exact. This is also based only on individual data and based on the queries of a random sample of 0.1% of users.

[SVG OMITTED]

[\[find in text\]](#)

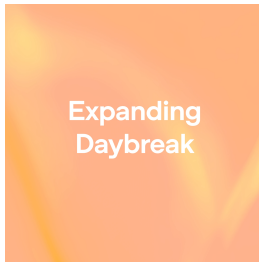
Keep reading

[View all](#)



OpenAI and Broadcom unveil LLM-optimized inference chip

CompanyJun 24, 2026



Daybreak: Tools for securing every organization in the world

SecurityJun 22, 2026



Samsung Electronics brings ChatGPT and Codex to employees

CompanyJun 21, 2026

LLM Judges aren't the shortcut you think

DOUG TURNBULL · 03 NOV 2025 · [SOURCE](#)

Is “LLM as a judge” overhyped? After years of implementing LLM judges for clients, I find more of them recognize the limitations.

LLM judges fail for the same reason any evaluation method fails: teams lack good data+evaluation to begin with. LLM judges aren't the shortcut teams hope. Instead, they are themselves a thing that needs data and monitoring to get use out of.

Let me walk through the pitfalls of LLM as a judge (for search) that you'll encounter.

LLM as a judge

When I talk about LLM as a judge, my focus is search. I mean creating a judgment list. For search that means labeling how relevant a document is for a query. How relevant is the product “garden trowel” for a `q=shovels`? Maybe this “garden trowel” is less relevant than “iron spade”?

In the ancient past, to improve the search, I would be limited to a tiny hand-labeled judgment list. Labeling by hand is expensive. Datasets drift out of date quickly. New use cases arise. So we constantly need humans to relabel.

These costs inspire teams to use LLMs to build judgments, They:

1. Start with a ground truth of human labels
2. Build a prompt to approximate that ground truth
3. Apply those prompts to go beyond the ground truth to new query / documents

If done well, we can sit at our laptop, tinker with relevance algorithms, and continue to tune without bother humans for non-stop relabeling.

Sounds promising. But teams often ignore many of the assumptions and nuance underlying the task of LLM labeling. Let's go through them, and we'll see at the end, maybe LLMs should play a different role in evaluation?

Problem one - LLM evals don't know what's engaging to users

Users' ever-evolving lizard brains drive conversions. And while optimizing for just lizard brains would itself be a nightmare, so would only focusing on an LLM's opinion.

LLMs do a better job approximating *human evaluations*. In search, we call these evaluations *explicit* judgments. They use human labelers' basic knowledge of the world to label a result as *topically relevant* to the query or not.

Topical relevance is only one kind of relevance. As an example, for a search of "articles about harry potter". The article is either factually about Harry Potter or its not. We might also consider other gradations:

- Is the article well written?
- Is it article from an authoritative source?
- Is the article about a tangential topic? Dumbledore vs Gandalf etc?

All of which relate to topicality, authority, and knowledge. An LLM excels at this — LLMs live in a world of facts and knowledge.

But LLMs don't have limbic systems. So their evaluations come with limitations.

In real search applications, "relevance" goes beyond topicality. I worked at Reddit on search. A search for "harry potter" might mean anything from a memes, to drama about JK Rowling, to every kind of juicy controversy, spicy post, or oddity. People search for "cybertruck" not for reviews but to see silly videos.

Then there's the more mundane. I found at Shopify, users preferred plain or black clothing over flashy items based on their clicks. Unless you tell an LLM this (because you've done the analysis) an LLM won't intrinsically catch this.

Often the "topical" part of search is relatively easy. But engaging search, that captures these human oddities, must go beyond world-knowledge to at least get *some* of a user's lizard brain.

Problem two - the last 10% of disagreement is the important stuff

Many teams get excited when they immediately see 70-80% agreement with human labelers. With a bit of tuning, they can inch closer to 90%.

In my experience, that last bit of human-LLM disagreement matters. Those are the non-obvious cases. Search deals with a big hard negatives challenge. Those examples that deceptively seem relevant naive search algorithms (or labelers), but are, in fact irrelevant. That last 10-30% might be what more careful labelers would catch. Or represented in clickstream based labels.

For example on this furniture dataset, a LLM doesn't understand that a "bistro table" is meant for **outdoor** use in US furniture lingo. That's not obvious to an LLM: The LLM imagines I'm opening a restaurant.

Simple algorithms often capture that first 80-90%. It's the 10-20% you need great labels for. So pay attention not just to coverage. But for which use cases.

A 90% coverage might be good regression tests, it might not help you go beyond easier wins.

Problem three - sneaky overfitting

So far, I've mentioned a few strange cases the LLM judge might miss: a bistro table is meant for outdoor usage. Users preferring darker clothing items.

The savvy prompt engineer will want to fill the prompt with examples to help the LLM perform better on these cases.

But beware overfitting to these particular failure cases.

With every new rule, you push the model's meager attention away from the general problem, and towards your exceptions. Layer example after example, and you'll find yourself with a brittle system focused excessively on specific use cases.

Overfitting with prompts can be sneaky. Playing whack-a-mole with your eval data can degrade generality. You can get perfect results on the eval set, only to expand to unseen queries, and find the gains no longer hold.

To do a good job, you need holdout data you're not tuning against. A set of judgments you don't peak at. Because peaking might cause you to sneak a change into the prompt, making the holdout less useful as an independent validation step.

Add rules and exceptions carefully, you'll be surprised how easy it is to pull an LLM away from its generality.

Problem four - LLMs only evaluate the document it sees

LLMs only evaluate the document it sees. IE:

Query: outdoor seating

Product Name: Bistro Table

Product Description: Enjoy this lovely bistro table on your porch...

Is this enough information for evaluating the product? Is it what a user sees?

Suddenly, you decide to add new information about the product

Query: outdoor seating

Product Name: Bistro Table

Product Description: Enjoy this lovely bistro table on your porch...

Category: Outdoors

What are we evaluating truly?

- The products relevance?
- The usefulness of a set of product properties for determining relevance?
- How comprehensively you're describing products to an LLM?

Really it gets down to a classic problem in search. There's perceived relevance - the relevance of the product, as it appears in your product page. Then there's actual relevance - the detailed, thoughtful understanding of your product by a user. A lot depends not just on true relevance, but how it's presented to users.

Problem five - Your LLM judge can't be trusted on novel use cases

I assume the goal of LLM as a judge in to align to human labelers. But rarely do those judgments comprehensively represent every use case.

Search metrics with human evals don't go up-and-to-the-right. They seesaw as teams reactively label, solve, label, solve ad infinitum. In my experience it's rare to find a team build a true, comprehensive human labeled evals covering every use case. Even if we did, we'd need to account for evolving search use cases.

The flow usually goes like this: we want to solve a problem (search by product name). So we capture some judgments for that use case. Relevance starts low. We work a bit to improve this use case and it goes up. Yay!

But the boss comes to our desk and suggests we have a new problem to solve: search of furniture by the room (living room, etc). We gather more labels. We know we're bad at this use case. So search metrics go back down. Sad 😞

We solve that use case, it goes back up to where it was before. Repeat. Up and down. See-saw.

This presents a problem for an LLM judge. If our human labels only focus on a subset of use cases, **we're also aligning our LLM judges on a subset of use cases.**

Our LLM judge generalizes to just a piece of the problem. The LLM judge can't be trusted on a new use case until we measure it's abilities in that use case.

So if you seesaw its human labeling, you must seesaw the alignment of the LLM judge to the use cases we capture in human eval. As we adopt a new use case, we need more human labels. We need to retest the LLM judge. And so on.

The task of human labeling does not end with LLM Judges. In fact its even more crucial.

How to LLM as a judge

Here's how I use LLM judges:

- To extend labels on use cases I already measure decently to new queries (because I can trust alignment of the LLM on this use case)
- As a safeguard against regressions

I trust the LLM judge to flag a regression on the use cases my human labels cover. The LLM is aligned and measured against these. I don't trust the LLM to discover new, unexpected ways users search. I look to the users for that, not LLMs.

Still many teams don't even do decent monitoring of topical relevance. There are plenty of obviously broken search engines. The ones that would benefit from an LLM judge, likely would benefit from rounds of human labeling first needed before blindly trusting an LLM judge.

We already have automated data labeling at home kids

If we have enough labels to evaluate LLM as a judge on a use case, we have enough to train an ML model to recover those labels. We can then see a better place LLM's fit into evaluation:

- LLMs focus on factual, topical parts of the problem. But miss other parts of the problem
- LLMs might see a subset of the content features
- Other non-LLM features might help us better evaluate relevance (embedding similarity, text matches, etc)
- Other ground-truths (besides human) might be a better definition of relevance (ie engagement based labels)

Instead of LLM as a Judge, **why not Learning to Rank as a judge?** We seem to have forgotten that ranking models provide value beyond directly deploying them to production. They can help guide more manual search tuning as well.

I'm becoming more of a fan of LLM-for-feature-generation over LLM as a judge. LLMs can cover the topical part of relevance. You can ask them "Which of these product names is more relevant to the query?". The downstream, traditional ML model is agnostic to where the features come from. They could be an embedding similarity, LLM decision, lexical similarity, or more.

That's much more my interest these days. When someone says "LLM as a judge" I nod and say yes let's build that, but then go on and probably look into using LLM features heavily in a larger model. LLMs can be very useful for rapid feature development: they're promptable and easy to play with.

If you're interested in this approach, check out this [blog post](#) and the corresponding [Haystack talk](#). Also a talk from Shopify [choosing cross encoders for evaluation over LLMs](#)

Enjoy!

Enjoy software.doug in training course form!



I hope you join me at [Cheat at Search with LLMs](#) to learn how to apply LLMs to search applications. Check out [this post](#) for a sneak preview.

Why I don't like the "staff engineer archetypes"

SEANGOEDECKE.COM RSS FEED · 03 MAY 2026 · [SOURCE](#)

The most influential piece of writing about staff engineers in the last decade has to be Will Larson's *Staff engineer archetypes*. He argues that the "staff engineer" title covers at least four very different roles: the team lead, the architect, the solver, and the right hand. This taxonomy gets cited a lot as advice for people who are trying to become effective staff engineers. For both of my promotions to staff engineer, my manager at the time linked me to the "staff engineer archetypes" and asked me to consider which of these archetypes I was aiming towards.

These archetypes definitely exist¹. However, I think it's bad practical advice to tell engineers to try and target them.

Archetypes do not make good goals

To see why, let's take the "team lead" archetype. Larson describes this as an informal technical leadership role: not necessarily an explicit authority figure, but someone who's good at scoping work, planning projects, and maintaining the kind of relationships (e.g. with other teams) needed

to successfully ship. If you want to fill this role, shouldn't you start trying to do these things? No! You don't become a technical leader by trying really hard to be a technical leader, much like you don't become a writer by trying really hard "to be a writer". You become a technical leader by *doing good technical work* until your skills and relationships emerge organically.

I wrote about this process in *Ratchet effects determine engineer reputation at large companies*. To get good at shipping large complex projects, you must start by shipping tiny pieces of work, until you're familiar enough with the system and you've built enough trust to take on slightly larger pieces. At each stage, if you do good work - "good work" here means "deliver shareholder value" - you will very naturally be given opportunities to work on more complex and important things. If you try to jump ahead, you're going to run into all kinds of problems:

- Important projects are usually assigned top-down, not bottom-up, so you'll either be trying to muscle out the planned engineering lead for a project or to pitch your own (complex, important) engineering task to senior management. Either way, good luck with that!
- You likely won't have a good enough relationship with senior management to know what their real priorities are.
- If you're not yet trusted to execute, you may get assigned "minders" (often current staff engineers) who will ghost-lead the project through you².
- You'll likely make poor technical decisions.

The other archetypes are like this as well. If you want to become a successful architect, you do not get there by studying software architecture in the abstract, because you can't design software you don't work on. The "solver" and "right hand" archetypes both rely on having an enormous amount of trust and influence. You can't aim for those archetypes directly, because trust and influence accumulate over time. In fact, the idea of "aiming for" a particular staff engineer archetype reflects a misunderstanding of what the staff engineer role is. What is the defining attribute of the staff engineering role, then?

What is a staff engineer?

A staff engineer has to be useful to the company. Of course, a senior or mid-level software engineer ought to be useful too, but all they *have* to do is execute on the job in front of them. If they end up not providing value (maybe their project turns out to be unimportant, or they don't get the support needed to succeed) that's their manager's problem, not theirs³. In contrast, staff engineers are expected to deliver value regardless: to make the project work, or to find something else useful to do if the project truly can't be salvaged.

This is an unfair expectation. Often projects really do fail through no fault of your own, and sometimes it just isn't possible to conjure useful work from thin air. That's actually by design: **the staff engineer role is supposed to be unfair**. Something many engineers don't realize is that all senior management and executive leadership roles are unfair too, in the same way. That's just part of the deal: executives are given power and great compensation, and in return they get thrown off the boat in bad weather⁴. "Staff engineer" is the first engineering role where you are held largely responsible for outcomes you don't control.

Developing a "staff engineer mindset" thus has very little to do with the archetypes. Instead, you should:

1. Develop the habit of constantly asking yourself "is this useful to the company" (and answering correctly).
2. Lose the habit of worrying about if you're being treated "fairly". Instead, try to think about your role in terms of incentives and consequences.

At the beginning, you won't look much like any of the staff engineer archetypes. You will look like being a level-headed engineer who can be trusted to move projects forward with a minimum of fuss, and who can be re-tasked to different work without complaining. You'll also look like someone who's paying a lot of attention to what their manager's actual priorities are, and who is thinking hard about how to fulfil those priorities (instead of their own goals).

If you do this for long enough, you'll eventually find yourself in one of the staff engineer archetypes. However, it probably won't be the one you're "aiming for". The whole point of being a staff engineer is that you're willing to fill whatever archetype the company needs at the time.

Final thoughts

In his original staff engineer post, Larson is pretty clear that these archetypes are more of an anthropological description of some of the varied niches staff engineers fill, not a how-to guide for succeeding in the role⁵. At the time, the "staff engineer" role was fairly new and people were still trying to figure out what it even meant. Pointing out that there were a few very different ways to succeed in the role was a genuinely novel observation.

The staff engineer archetypes are a good list of ways an engineer can be very useful to their organization - but only once they've built a deep relationship of trust with their organization's leadership. Advice on how to succeed as a staff engineer should be about **how to build that trust**, not about what to do once you have it.

1. One caveat that is too pedantic for the body of the post: each tech company has a different structure of roles. Some don't have the formal "staff" title at all, while others have "staff" as a fairly early rung on the ladder and a panoply of "senior staff", "senior principal staff", and so on roles above it. Like all "staff engineer" discourse, this post is not about the word itself but about the point in the engineering job ladder where progression becomes significantly more difficult.

↩

2. Impressing your VP's trusted lieutenants can actually be a good way to build trust in the medium-term, but you'd better hope you've built enough understanding of the system to do it right. If this process goes badly, your reputation in the org might be torched for years.

↩

3. In theory, at least. In practice it's always better to be useful (again, in the sense of "delivering shareholder value").

↩

4. This is why very senior leadership sometimes seem so unempathetic towards engineering complaints: their work environment operates by very different rules and norms to that of most engineers. I keep meaning to try and write about this and never succeeding. This draft is the closest thing I have to a deeper exploration of the point.

↩

5. For the record, my how-to guides are [here](#) and [here](#).

↩

Giving LLMs a personality is just good engineering

SEANGOEDECKE.COM RSS FEED · 03 MAR 2026 · [SOURCE](#)

AI skeptics often argue that current AI systems shouldn't be so human-like. The idea - most recently expressed in this [opinion piece](#) by Nathan Beacom - is that language models should explicitly be tools, like calculators or search engines. Although they *can* pretend to be people, they shouldn't, because it encourages users to overestimate AI capabilities and (at worst) slip into [AI psychosis](#). Here's a representative paragraph from the piece:

In sum, so much of the confusion around making AI moral comes from fuzzy thinking about the tools at hand. There is something that Anthropic could do to make its AI moral, something far more simple, elegant, and easy than what Askell is doing. Stop calling it by a human name, stop dressing it up like a person, and don't give it the functionality to simulate personal relationships, choices, thoughts, beliefs, opinions, and feelings that only persons really possess. Present and use it only for what it is: an extremely impressive statistical tool, and an imperfect one. If we all used the tool accordingly, a great deal of this moral trouble would be resolved.

So why do Claude and ChatGPT act like people? According to Beacom, AI labs have built human-like systems because AI lab engineers are trying to hoodwink users into emotionally investing in the models, or because they're delusional true believers in AI personhood, or some other foolish reason. This is wrong. AI systems are human-like because **that is the best way to build a capable AI system.**

Modern AI models - whether designed for chat, like OpenAI's GPT-5.2, or designed for long-running agentic work, like Claude Opus 4.6 - do not naturally emerge from their oceans of training data. Instead, when you train a model on raw data, you get a "base model", which is not very useful by itself. You cannot get it to write an email for you, or proofread your essay, or review your code.

The base model is a kind of mysterious gestalt of its training data. If you feed it text, it will sometimes continue in that vein, or other times it will start outputting pure gibberish. It has no problem producing code with giant security flaws, or horribly-written English, or racist screeds - all of those things are represented in its training data, after all, and the base model does not judge. It simply outputs.

To build a *useful* AI model, you need to journey into the wild base model and stake out a region that is amenable to human interests: both ethically, in the sense that the model won't abuse its users, and practically, in the sense that it will produce correct outputs more often than incorrect ones. What this means in practice is that **you have to give the model a personality** during post-training¹.

Human beings are capable of almost any action at any time. But we only take a tiny subset of those actions, because that's the kind of people we are. I could throw my cup of coffee all over the wall right now, but I don't, because I'm not the kind of person who needlessly makes a mess². AI systems are the same. Claude could respond to my question with incoherent racist abuse - the base model is more than capable of those outputs - but it doesn't, because that's not the kind of "person" it is.

In other words, human-like personalities are not imposed on AI tools as some kind of marketing ploy or philosophical mistake. Those personalities are the medium via which the language model can become useful at all. This is why it's surprisingly tricky to "just" change a language model's personality or opinions: because you're navigating through the near-infinite manifold of the base model. You may be able to control which direction you go, but you can't control what you find there³.

When AI people talk about LLMs having personalities, or wanting things, or even having souls⁴, these are technical terms, like the "memory" of a computer or the "transmission" of a car. You simply cannot build a capable AI system that "just acts like a tool", because the model is trained on *humans* writing to and about other *humans*. You need to prime it with some kind of personality (ideally that of a useful, friendly assistant) so it can pull from the helpful parts of its training data instead of the horrible parts.

1. This is all pretty well understood in the AI space. Anthropic wrote a [recent paper](#) about it where they cite similar positions going all the way back to 2022. But for some reason it's not yet penetrated into communities that are more skeptical of AI.

↩

2. You could explain this in terms of "the stories we tell ourselves". Many people (though not all) think that human identities are narratively constructed.

↩

3. I wrote about this last year in *Mecha-Hitler, Grok, and why it's so hard to give LLMs the right personality*. A little nudge to change Grok's views on South African internal politics can cause it to start calling itself "Mecha-Hitler".

↩

4. I have long believed that Claude "feels better" to use than ChatGPT because it has a more coherent persona (due mainly to Amanda Askell's work on its "soul"). My guess is that if you tried to make a "less human" version of Claude, it would become rapidly less capable.

↩